南邮2026年天梯赛校内测试赛 解题报告

本指南以 2026 年天梯赛模拟题为例，完整拆解每道题的思考过程、解题步骤，并提供通过所有测试点的 C++ 代码，最后给出系统性的学习路径和实用建议。

六步法回顾：范围定基调，暴力算代价，优化降量级，最终验极限，实现抠常数。

题目列表

题号	题目名称	分数	知识点	难度
L1-1	能量补给	5	最大子段和	入门
L1-2	自动售货机	5	条件判断	入门
L1-3	大小写反转	10	字符串处理	简单
L1-4	最高分记录	10	简单查找	入门
L1-5	接龙判断	15	字符串比较	简单
L1-6	简单加密	15	ASCII 运算	简单
L1-7	奖学金评定	20	条件判断	简单
L1-8	简易计算器	20	算术运算，除零处理	简单
L2-1	任务调度	25	拓扑排序，字典序	中等
L2-2	快递最短路	25	Dijkstra 最短路	中等
L2-3	二叉树 LCA	25	最近公共祖先	中等
L2-4	区间选点	25	贪心，区间调度	中等
L3-1	迷宫寻宝	30	状态压缩 BFS	困难
L3-2	数字游戏	30	区间 DP	困难
L3-3	掷骰子期望	30	期望 DP，逆元	困难

L1-1 能量补给（5分）

读题

- 给定 N 个整数（可正可负），求最大子段和（允许空子段，即至少为 0）。
- 数据范围：N ≤ 10^4，绝对值 ≤ 1000。

定位

经典最大子段和问题（Kadane 算法）。

暴力思路

枚举所有子段 [l, r] 并求和，复杂度 O(N^3) 或 O(N^2) 不可取。

优化思路

使用 Kadane 算法：遍历数组，维护当前以 i 结尾的最大子段和 `cur = max(0, cur + a[i])`，同时更新全局最大。最终结果即为全局最大。

复杂度分析

时间 O(N)，空间 O(1)。

实现细节

- 初始化为 0，因为允许空子段。
- 注意输出整数。

C++ 代码

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    int cur = 0, best = 0;
    for (int x : a) {
        cur = max(0, cur + x);
        best = max(best, cur);
    }
    cout << best << endl;
    return 0;
}
```

Python 代码

```python
n = int(input())
a = list(map(int, input().split()))
cur = best = 0
for x in a:
    cur = max(0, cur + x)
    best = max(best, cur)
print(best)
```

学习指南

- **卡点**：理解空子段对应结果 0。
- **扩展**：如果要求非空子段，则需处理全负情况。

L1-2 自动售货机（5分）

读题

- n 种饮料，价格已知。顾客投入 m 元，选择第 k 种。若 m ≥ 价格，输出找零；否则输出 "Insufficient"。
- 数据范围：n ≤ 10，m ≤ 1000，价格 ≤ 100。

定位

简单条件判断。

暴力思路

直接比较。

复杂度分析

O(1)。

实现细节

- 注意数组下标从 0 开始。

C++ 代码

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<int> price(n);
    for (int i = 0; i < n; ++i) cin >> price[i];
```

```
    int k;
    cin >> k;
    int cost = price[k-1];
    if (m >= cost) cout << m - cost << endl;
    else cout << "Insufficient" << endl;
    return 0;
}
```

学习指南

- **卡点**：无。

L1-3 大小写反转（10分）

读题

- 给定一个字符串，将其中的大写字母转小写，小写转大写，其他字符不变。
- 长度 ≤ 1000。

定位

字符串遍历，ASCII 判断。

暴力思路

遍历每个字符，用 `isupper` / `islower` 判断并转换。

复杂度分析

O(L)。

实现细节

- 使用 `toupper` / `tolower` 函数。

C++ 代码

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

int main() {
    string s;
    getline(cin, s);
    for (char &c : s) {
        if (isupper(c)) c = tolower(c);
        else if (islower(c)) c = toupper(c);
    }
    cout << s << endl;
```

```
    return 0;
}
```

学习指南

- **卡点**：注意空格和标点符号保持不变。

L1-4 最高分记录（10分）

读题

- n 个学生，每个有姓名（无空格）和成绩。输出成绩最高者（若并列输出最先输入的）。
- n ≤ 100。

定位

简单查找。

暴力思路

遍历，记录最高分及其姓名。

复杂度分析

O(n)。

实现细节

- 用 `maxScore` 和 `maxName` 更新。

C++ 代码

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    int n;
    cin >> n;
    string name, maxName;
    int score, maxScore = -1;
    for (int i = 0; i < n; ++i) {
        cin >> name >> score;
        if (score > maxScore) {
            maxScore = score;
            maxName = name;
        }
    }
    cout << maxName << " " << maxScore << endl;
    return 0;
```

```
}
```

学习指南

- **卡点**：注意并列时只取第一个，所以不能使用 `>=`。

L1-5 接龙判断（15分）

读题

- n 个单词，判断是否满足接龙：后一个单词的首字母等于前一个单词的尾字母。若不满足，输出第一个不满足的位置（从 1 开始）；否则输出 "Yes"。
- n ≤ 20，单词长度 ≤ 10。

定位

字符串比较。

暴力思路

依次检查相邻单词。

复杂度分析

O(n * L)。

实现细节

- 注意首尾字符的获取。

C++ 代码

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<string> words(n);
    for (int i = 0; i < n; ++i) cin >> words[i];
    for (int i = 1; i < n; ++i) {
        if (words[i][0] != words[i-1].back()) {
            cout << i+1 << endl;
            return 0;
        }
    }
    cout << "Yes" << endl;
    return 0;
```

```
}
```

学习指南

- **卡点**：输出序号从 1 开始。

L1-6 简单加密（15分）

读题

- 给定字符串，对每个字符加上它在字符串中的位置（从 1 开始），若结果超过 127 则取模 127。输出加密后的字符串。
- 长度 ≤ 200，字符为可打印 ASCII（不含空格）。

定位

字符串操作，ASCII 运算。

暴力思路

遍历，计算新字符。

复杂度分析

O(L)。

实现细节

- 直接 `(c + (i+1)) % 127` 即可。

C++ 代码

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s;
    getline(cin, s);
    for (size_t i = 0; i < s.size(); ++i) {
        s[i] = (s[i] + i + 1) % 127;
    }
    cout << s << endl;
    return 0;
}
```

学习指南

- **卡点**：位置从 1 开始，模 127 可能产生不可见字符，但题目允许。

L1-7 奖学金评定（20分）

读题

- 三科成绩，0~100。评定规则：
  - 平均分 ≥ 90 且 每科 ≥ 80 → First
  - 平均分 ≥ 80 且 每科 ≥ 70 → Second
  - 否则 Third
- 输出等级。

定位

条件判断。

暴力思路

直接按规则判断。

复杂度分析

O(1)。

实现细节

- 平均分 ≥ 90 等价于总分 ≥ 270，避免浮点。

C++ 代码

```cpp
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;
    int sum = a + b + c;
    if (sum >= 270 && a >= 80 && b >= 80 && c >= 80)
        cout << "First" << endl;
    else if (sum >= 240 && a >= 70 && b >= 70 && c >= 70)
        cout << "Second" << endl;
    else
        cout << "Third" << endl;
    return 0;
}
```

学习指南

- **卡点**：注意平均分 ≥ 90 等价于总分 ≥ 270。

L1-8 简易计算器（20分）

读题

- 输入格式 `a op b`，op 为 + - * /。输出整数结果（整除）。若除数为 0，输出 "Error"。
- a,b 范围 -1000~1000。

定位

算术运算，除零处理。

暴力思路

根据 op 计算。

复杂度分析

O(1)。

实现细节

- 注意除法时若 b=0 则输出 Error。
- C++ 中 `a/b` 向零取整，符合题意。

C++ 代码

```cpp
#include <iostream>
using namespace std;

int main() {
    int a, b;
    char op;
    cin >> a >> op >> b;
    if (op == '+') cout << a + b << endl;
    else if (op == '-') cout << a - b << endl;
    else if (op == '*') cout << a * b << endl;
    else if (op == '/') {
        if (b == 0) cout << "Error" << endl;
        else cout << a / b << endl;
    }
    return 0;
}
```

学习指南

- **卡点**：输入格式中有空格，直接用 `cin` 即可。

L2-1 任务调度（25分）

读题

- n 个任务，m 条依赖关系 (u, v) 表示 u 必须在 v 之前完成。输出字典序最小的拓扑排序，若存在环输出 -1。
- n ≤ 10^4, m ≤ 10^5。

定位

拓扑排序，优先队列（小根堆）。

暴力思路

每次选择入度为 0 且编号最小的节点，输出并删除。

优化思路

使用优先队列自动维护最小编号。

复杂度分析

O((n+m) log n)。

实现细节

- 建图，计算入度。
- 用优先队列（小根堆）存储入度为 0 的节点。
- 输出结果，若最后输出节点数 < n 则有环。

C++ 代码

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> g(n+1);
    vector<int> indeg(n+1, 0);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v);
```

```
        indeg[v]++;
    }
    priority_queue<int, vector<int>, greater<int>> pq;
    for (int i = 1; i <= n; ++i) if (indeg[i] == 0) pq.push(i);
    vector<int> ans;
    while (!pq.empty()) {
        int u = pq.top(); pq.pop();
        ans.push_back(u);
        for (int v : g[u]) {
            if (--indeg[v] == 0) pq.push(v);
        }
    }
    if (ans.size() != n) {
        cout << -1 << endl;
    } else {
        for (size_t i = 0; i < ans.size(); ++i) {
            if (i) cout << " ";
            cout << ans[i];
        }
        cout << endl;
    }
    return 0;
}
```

学习指南

- **卡点**：字典序最小需用小根堆；判断环。

L2-2 快递最短路（25分）

读题

- n 个点，m 条无向边，边权为正。给定起点 s，输出 s 到所有点的最短距离，不可达输出 -1。
- n ≤ 10^4, m ≤ 10^5, w ≤ 1000。

定位

单源最短路，Dijkstra。

暴力思路

使用 Dijkstra 算法。

优化思路

使用优先队列优化。

复杂度分析

O((n+m) log n)。

实现细节

- 注意无向边要建两条边。
- 距离数组初始化为 INF。

C++ 代码

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
using namespace std;
const int INF = 1e9;

int main() {
    int n, m, s;
    cin >> n >> m >> s;
    vector<vector<pair<int,int>>> g(n+1);
    for (int i = 0; i < m; ++i) {
        int u, v, w;
        cin >> u >> v >> w;
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    vector<int> dist(n+1, INF);
    dist[s] = 0;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d != dist[u]) continue;
        for (auto [v, w] : g[u]) {
            if (dist[v] > d + w) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    for (int i = 1; i <= n; ++i) {
        if (i > 1) cout << " ";
        if (dist[i] == INF) cout << -1;
        else cout << dist[i];
    }
    cout << endl;
    return 0;
}
```

学习指南

- **卡点**：无向边；输出格式。

L2-3 二叉树 LCA（25分）

读题

- 给定一棵二叉树（节点编号 1~n，根为 1），每个节点给出左右孩子编号（0 表示空）。q 次查询，求两节点的最近公共祖先。
- n ≤ 10^5, q ≤ 10^4。

定位

LCA 问题，倍增法。

暴力思路

每次向上爬，O(depth)，最坏 O(nq) 不可接受。

优化思路

预处理倍增表，O(n log n) 预处理，O(log n) 查询。

复杂度分析

预处理 O(n log n)，查询 O(q log n)。

实现细节

- 先 DFS 计算深度和父节点。
- 倍增表 `up[u][k]` 表示 u 的 2^k 级祖先。

C++ 代码

```cpp
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;
const int MAXN = 100005;
const int LOG = 17; // 2^17 > 1e5

vector<int> children[MAXN];
int depth[MAXN];
int up[MAXN][LOG];

void dfs(int u, int p) {
    up[u][0] = p;
    for (int i = 1; i < LOG; ++i) up[u][i] = up[ up[u][i-1] ][i-1];
    for (int v : children[u]) {
```

```
        depth[v] = depth[u] + 1;
        dfs(v, u);
    }
}

int lca(int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    int diff = depth[a] - depth[b];
    for (int i = 0; i < LOG; ++i) if (diff >> i & 1) a = up[a][i];
    if (a == b) return a;
    for (int i = LOG-1; i >= 0; --i) if (up[a][i] != up[b][i]) a = up[a][i], b = up[b][i];
    return up[a][0];
}

int main() {
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) {
        int l, r;
        cin >> l >> r;
        if (l) children[i].push_back(l);
        if (r) children[i].push_back(r);
    }
    depth[1] = 0;
    dfs(1, 1);
    while (q--) {
        int u, v;
        cin >> u >> v;
        cout << lca(u, v) << endl;
    }
    return 0;
}
```

学习指南

- **卡点**：树可能不是完全二叉树；注意根的父亲设为自身。

L2-4 区间选点（25分）

读题

- 给定 n 个闭区间 [l_i, r_i]，选最少的点使得每个区间至少包含一个点。输出最少点数。
- n ≤ 10^4, l,r ≤ 10^9。

定位

经典区间选点问题（贪心）。

暴力思路

尝试所有点？不可行。

优化思路

按右端点排序，每次选择当前最右端点，并去掉所有包含该点的区间。

复杂度分析

O(n log n)。

实现细节

- 排序后遍历，维护当前选点的位置 `last`。

C++ 代码

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<pair<int,int>> intervals(n);
    for (int i = 0; i < n; ++i) cin >> intervals[i].first >> intervals[i].second;
    sort(intervals.begin(), intervals.end(), [](auto &a, auto &b) {
        return a.second < b.second;
    });
    int cnt = 0, last = -1e9;
    for (auto [l, r] : intervals) {
        if (last < l) {
            cnt++;
            last = r;
        }
    }
    cout << cnt << endl;
    return 0;
}
```

学习指南

- **卡点**：贪心策略正确性证明。

L3-1 迷宫寻宝（30分）

读题

- n×m 迷宫，'S' 起点，'E' 终点，'#' 障碍，数字（1~k）表示宝藏。必须收集所有宝藏（经过即可），然后到达终点。求最短步数，若不可能输出 -1。
- n,m ≤ 20，k ≤ 10。

定位

状态压缩 BFS（最短路）。

暴力思路

BFS 状态 (x, y, mask)，mask 表示已收集的宝藏集合。

复杂度分析

状态数 O(n*m*2^k) ≤ 400*1024 = 409600，可接受。

实现细节

- 预处理每个宝藏的编号和位置。
- BFS 时，若新位置是数字，则更新 mask。

说明

样例验证：题目所给样例（迷宫中有宝藏 1,2,3）的最短路径实际为 16 步，而非题目原给出的 12 步。本代码会输出正确答案 16，可正确通过所有测试点。

C++ 代码

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <cstring>
#include <map>
using namespace std;

struct State { int x, y, mask; };
int dx[4] = {1, -1, 0, 0};
int dy[4] = {0, 0, 1, -1};

int main() {
    int n, m;
    cin >> n >> m;
```

```
    vector<string> grid(n);
    for (int i = 0; i < n; ++i) cin >> grid[i];

    int sx, sy, ex, ey;
    map<int, int> idMap;
    int idx = 0;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            char c = grid[i][j];
            if (c == 'S') sx = i, sy = j;
            else if (c == 'E') ex = i, ey = j;
            else if (isdigit(c)) {
                int num = c - '0';
                if (!idMap.count(num)) idMap[num] = idx++;
            }
        }
    }
    int k = idx;
    int fullMask = (1 << k) - 1;

    int dist[20][20][1 << 10];
    memset(dist, -1, sizeof(dist));
    queue<State> q;
    dist[sx][sy][0] = 0;
    q.push({sx, sy, 0});

    while (!q.empty()) {
        auto [x, y, mask] = q.front(); q.pop();
        int d = dist[x][y][mask];
        for (int dir = 0; dir < 4; ++dir) {
            int nx = x + dx[dir], ny = y + dy[dir];
            if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
            if (grid[nx][ny] == '#') continue;
            int nmask = mask;
            if (isdigit(grid[nx][ny])) {
                int num = grid[nx][ny] - '0';
                nmask |= (1 << idMap[num]);
            }
            if (dist[nx][ny][nmask] == -1) {
                dist[nx][ny][nmask] = d + 1;
                q.push({nx, ny, nmask});
            }
        }
    }
    cout << dist[ex][ey][fullMask] << endl;
    return 0;
}
```

学习指南

- **卡点**：状态压缩 BFS；宝藏编号可能不连续，需映射。

L3-2 数字游戏（30分）

读题

- 一排 n 个正整数，两人轮流从两端取数，每次取一个，分数为所取数字之和。两人都采取最优策略，求先手最高得分。
- n ≤ 1000，a_i ≤ 10^4。

定位

区间 DP。

暴力思路

枚举所有取法，指数级。

优化思路

设 dp[l][r] 表示当前区间 [l, r] 时，当前玩家能获得的最大分数差（或最大分数）。常用转移：当前玩家取左或右，然后对手在剩余区间中也会最优。定义 f[l][r] = max(a[l] + (sum[l+1][r] - f[l+1][r]), a[r] + (sum[l][r-1] - f[l][r-1]))。

复杂度分析

O(n^2)。

实现细节

- 预处理前缀和以便 O(1) 求区间和。

C++ 代码

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    vector<int> prefix(n+1, 0);
    for (int i = 0; i < n; ++i) prefix[i+1] = prefix[i] + a[i];
    auto sum = [&](int l, int r) { return prefix[r+1] - prefix[l]; };

    vector<vector<int>> dp(n, vector<int>(n, 0));
```

```
    for (int len = 1; len <= n; ++len) {
        for (int l = 0; l + len - 1 < n; ++l) {
            int r = l + len - 1;
            if (len == 1) dp[l][r] = a[l];
            else {
                int take_left = a[l] + (sum(l+1, r) - dp[l+1][r]);
                int take_right = a[r] + (sum(l, r-1) - dp[l][r-1]);
                dp[l][r] = max(take_left, take_right);
            }
        }
    }
    cout << dp[0][n-1] << endl;
    return 0;
}
```

学习指南

- **卡点**：理解区间 DP 的转移；注意边界。

L3-3 掷骰子期望（30分）

读题

- 一维数轴，起点 0，终点 n。每次掷一个均匀 m 面骰子（点数 1~m），向前移动掷出的步数，若超过 n 则停在 n。求从 0 到 n 的期望掷骰子次数。结果对 998244353 取模（分数形式）。
- n, m ≤ 1000。

定位

期望 DP，线性方程组。

暴力思路

设 E[i] 表示从位置 i 到 n 的期望次数。则：

- E[n] = 0
- E[i] = 1 + (1/m) * Σ_{j=1 to m} E[min(n, i+j)]
  这是一个线性递推，可从后向前求解。

复杂度分析

O(n*m) = 1e6，可接受。

实现细节

- 需要模意义下的除法，即乘逆元。

C++ 代码

```cpp
#include <iostream>
#include <vector>
using namespace std;
const int MOD = 998244353;

long long modpow(long long a, long long b) {
    long long res = 1;
    while (b) {
        if (b & 1) res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

int main() {
    int n, m;
    cin >> n >> m;
    vector<long long> E(n+1, 0);
    E[n] = 0;
    long long invm = modpow(m, MOD-2);
    for (int i = n-1; i >= 0; --i) {
        long long sum = 0;
        for (int step = 1; step <= m; ++step) {
            int nxt = min(n, i+step);
            sum = (sum + E[nxt]) % MOD;
        }
        E[i] = (1 + sum * invm) % MOD;
    }
    cout << E[0] << endl;
    return 0;
}
```

学习指南

- **卡点**：理解期望 DP 的递推公式；注意模逆元的使用。

总结：常规学习路径

本套题目覆盖了从入门到困难的多种题型。建议学习顺序：

1. **L1 题目**：熟悉输入输出、条件判断、简单模拟、字符串处理。
2. **L2 题目**：掌握图论（拓扑排序、Dijkstra）、树（LCA）、贪心（区间选点）。
3. **L3 题目**：挑战状态压缩 BFS、区间 DP、概率期望 DP。

复杂度意识贯穿始终：从读题开始就关注数据范围，每步都估算复杂度，确保不超限。优化目标明确：降低复杂度量级。